

Start

pdf_parse.py

```
import re

def extract_process_info(text):
    Procedure = ""
    Time = ""
    Dur = ""
    match = re.search(r"Procedure\s*:\s*(.*?)\s*(?=Time\s*:$)", text, re.IGNORECASE |
re.MULTILINE)
    if match:
        Procedure = match.group(1).strip()
    match = re.search(r"Time\s*:\s*(.*?)\s*(?=Mode\s*:$)", text, re.IGNORECASE |
re.MULTILINE)
    if match:
        Time = match.group(1).strip()
    match = re.search(r"Dur\s*:\s*(.*?)\s*(?=Priority\s*:$)", text, re.IGNORECASE |
re.MULTILINE)
    if match:
        Dur = match.group(1).strip()
    return Procedure, Time, Dur

def extract_status(text):
    status_data = {}
    lines = text.splitlines()
    for line in lines:
        match = re.match(r"^s*(\d+\.\d+)\s+.*?\b(Success|Failure)\s*$", line, re.IGNORECASE)

        if match:
            STEP_ID = match.group(1)
            STATUS = match.group(2).lower()

            if STATUS not in status_data:
                status_data[STATUS] = []

            status_data[STATUS].append(STEP_ID)

    return status_data
```

main.py

```
import fitz
from pdf_parser import extract_process_info, extract_status
pdf = fitz.open("input/input.pdf")
page = pdf[0]
text = page.get_text()
Procedure, Time, Dur = extract_process_info(text)
status_data = extract_status(text)
process_data = {
    "Procedure": Procedure,
    "Time": Time,
    "Dur": Dur,
    "status": status_data
}
print(process_data)
pdf.close()
```

output

```
{
  'Procedure': 'abbnm',
  'Time': '31-dec-2025 11:44:13.103',
  'Dur': '203.96(s)',
  'status': {
    'success': ['01.0001'],
    'failure': ['01.0002']
  }
}
```

Only do STEP 6 now.

Keep your working pdf_parser.py exactly as it is.

Change only main.py to:

```

import fitz
from pdf_parser import extract_process_info, extract_status
pdf = fitz.open("input/input.pdf")
all_text = ""
for page in pdf:
    all_text = all_text + page.get_text() + "\n"
print("Total pages:", len(pdf))
print("Total text length:", len(all_text))
pdf.close()

```

and then the putput should be -

If you get something like:

Total pages: ...

Total text length: ...

with **no error**, then Step 6 is complete.

Total pages: 50

Total text length: 123456

STEP 7 — Check how many Procedures are in the PDF

For this step, **do not change pdf_parser.py**.

Only change main.py.

Replace its contents with:

```

import fitz
import re
pdf = fitz.open("input/input.pdf")
all_text = ""
for page in pdf:
    all_text = all_text + page.get_text() + "\n"
procedures = re.findall(r"Procedure\s*:", all_text, re.IGNORECASE)
print("Total pages:", len(pdf))
print("Total procedures found:", len(procedures))
pdf.close()

```

OUTPUT

Total pages: 50

Total procedures found: 3

STEP 8 — Separate every Procedure

Change main.py

For this step, temporarily replace main.py with:

```
import fitz
import re
pdf = fitz.open("input/input.pdf")
all_text = ""
for page in pdf:
    all_text = all_text + page.get_text() + "\n"
procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]
print("Total pages:", len(pdf))
print("Total procedures:", len(procedure_blocks))
for i, block in enumerate(procedure_blocks, start=1):
    print("Procedure", i)
    print(block[:200])
    print("-----")
pdf.close()
```

Expected output

Total pages: 50

Total procedures: 20

Procedure 1

Procedure : abbnm Time:31-dec-2025 11:44:13.103 ...

Source: ...

Dur:203.96(s) ...

STEP ID ...

01.0001 ...

01.0002 ...

Procedure 2

Procedure : ...

...

⚠️ One important check

If your PDF has some Procedure: text inside random/unrelated content, this simple split could create an extra block.

If the number of blocks from Step 8 is different from the number you got in Step 7, stop and don't continue.

But if:

Step 7:

Total procedures found: 20

and Step 8 gives:

Total procedures: 20

then:

STEP 8 is successful. ✅

Good. Let's proceed to **Step 9**.

The goal now is:

Take every procedure block from Step 8 and use the extraction functions that we already tested successfully.

We will **not change pdf_parser.py**. It is already working.

STEP 9 — Extract data from every procedure block

Replace main.py with this:

```
import fitz
import re
from pdf_parser import extract_process_info, extract_status
pdf = fitz.open("input/input.pdf")
all_text = ""
for page in pdf:
    all_text = all_text + page.get_text() + "\n"
procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
    status_data = extract_status(block)
    process_data = {
        "Procedure": Procedure,
        "Time": Time,
```

```
"Dur": Dur,  
"status": status_data  
}
```

```
all_processes.append(process_data)
```

```
print("Total procedures:", len(all_processes))  
for process in all_processes:  
    print(process)  
pdf.close()
```

What this does

For every block:

Procedure 1

↓

extract_process_info()

↓

Procedure + Time + Dur

↓

extract_status()

↓

success + failure STEP IDs

↓

create dictionary

Then it repeats for:

Procedure 2

Procedure 3

Procedure 4

...

and stores everything in:

all_processes

which is a list.

Expected output

The beginning should look approximately like:

Total procedures: 20

```
{'Procedure': 'abbnm',
```

```
'Time': '31-dec-2025 11:44:13.103',
```

```
'Dur': '203.96(s)',
```

```
'status': {  
  'success': ['01.0001'],  
  'failure': ['01.0002']  
}}
```

```
{'Procedure': 'another_procedure',  
 'Time': '...',  
 'Dur': '...',  
 'status': {  
   'success': [...],  
   'failure': [...]  
 }}
```

The important checks

For the first procedure, verify that you still get:

```
{  
  'Procedure': 'abbnm',  
  'Time': '31-dec-2025 11:44:13.103',  
  'Dur': '203.96(s)',  
  'status': {  
    'success': ['01.0001'],  
    'failure': ['01.0002']  
  }  
}
```

Step 5:

one procedure → one dictionary 

Step 8:

complete PDF → separate procedure blocks 

Step 9:

each block → one dictionary

STEP 10 – ANOTHER PDF

STEP 10 — Create output.json

This document contains the complete Step 10 instructions, including what to create, what not to create, the exact code, expected output, checks, and possible errors.

What we already have

At the end of Step 9, the program has a Python variable called **all_processes**. It contains all the procedure dictionaries.

```
[
    {
        "Procedure": "abbnm",
        "Time": "31-dec-2025 11:44:13.103",
        "Dur": "203.96(s)",
        "status": {
            "success": ["01.0001"],
            "failure": ["01.0002"]
        }
    },
    ...
]
```

10.1 — Do NOT create the JSON file manually

You do NOT need to right-click and create a JSON file, create output.json yourself, write {} inside it, or copy anything into it. Python will automatically create **output.json** when the program runs.

```
with open("output.json", "w") as file:
```

10.2 — Add the json import

At the top of main.py, add:

```
import json
```

The imports should therefore be:

```
import fitz
import re
import json

from pdf_parser import extract_process_info, extract_status
```

10.3 — Complete main.py for Step 10

Delete the current contents of main.py and use this complete version:

```
import fitz
import re
import json

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
```

```

status_data = extract_status(block)

process_data = {
    "Procedure": Procedure,
    "Time": Time,
    "Dur": Dur,
    "status": status_data
}

all_processes.append(process_data)

with open("output.json", "w") as file:
    json.dump(all_processes, file, indent=4)

print("Total procedures:", len(all_processes))
print("JSON file created successfully.")

pdf.close()

```

10.4 — Run main.py

Click Run. You should see output similar to:

```

Total procedures: 20
JSON file created successfully.

```

The number 20 is only an example. Your PDF may have a different number of procedures. The important part is that there is no error and the message **JSON file created successfully.** appears.

10.5 — Check the project folder

After running the program, your project should contain a new file:

```

samarth/
■
■■■ main.py
■■■ pdf_parser.py
■■■ input/
■   ■■■ input.pdf
■■■ output.json      ← Python creates this
■■■ PyMuPDF-1.18.18

```

Again: you do not create output.json manually. Python creates it.

10.6 — Open output.json

Double-click output.json. It should contain all the procedures. The first one should look approximately like:

```

[
  {
    "Procedure": "abbnm",
    "Time": "31-dec-2025 11:44:13.103",
    "Dur": "203.96(s)",
    "status": {
      "success": [
        "01.0001"
      ],
      "failure": [
        "01.0002"
      ]
    }
  },
  {
    "Procedure": "another_process",
    "Time": "...",
    "Dur": "...",
    "status": {
      "success": [
        "02.0001"
      ],
      "failure": [

```

```

    ]
  }
]

```

JSON uses double quotes such as "Procedure". That is normal. Earlier Python print output used single quotes because that was Python's dictionary representation. `json.dump()` writes proper JSON with double quotes.

10.7 — If output.json already exists

That is okay. The code uses `open("output.json", "w")`. The "w" mode means Python writes the new result and replaces the old contents. You do not need to delete `output.json` manually.

10.8 — Possible errors

Error 1: No module named 'fitz'

This means the PyMuPDF setup is not available. In your current project this should not happen because `fitz` has already worked.

Error 2: No module named 'pdf_parser'

Check that the Python file name is exactly **pdf_parser.py** and that the import is:

```
from pdf_parser import extract_process_info, extract_status
```

If your actual file has a different name, the import must match that name.

Error 3: NameError: name 'json' is not defined

Make sure this line is present at the top:

```
import json
```

Error 4: PermissionError

This can happen if `output.json` is open in another program and prevents Python from writing to it. Close `output.json` and run `main.py` again.

Error 5: Program runs but output.json is empty

If the program prints **Total procedures: 0**, do not continue. That means procedure detection failed earlier. The JSON may be created but would contain:

```
[]
```

That is not a correct project result. Do not manually edit the JSON; the parser needs to be fixed.

Error 6: JSON is created but information is wrong

For example, if Procedure, Time, Dur, or status values are empty or procedures are missing, do not manually edit the JSON. The JSON is only the output; the parser that produced it must be checked.

10.9 — What counts as SUCCESS for Step 10?

- The program runs without an error.
- The message 'JSON file created successfully.' appears.
- A new `output.json` appears automatically in the project folder.
- The number of procedures is the same as the number verified in Step 7/8. For example: Step 7 = 20, Step 8 = 20, Step 10 = 20.
- Open `output.json` and verify the first procedure contains Procedure, Time, Dur, and status with success/failure STEP IDs.

IMPORTANT — Do not move to Step 11 immediately

After Step 10, output.json is the first major final output. Before doing anything else, verify that the first procedure in output.json matches the corresponding procedure in the PDF. Then check several procedures, procedure counts, missing statuses, STEP IDs, and other values. Only after verification should you continue to Step 11.

Printable sequence

```
STEP 6
Read all pages
↓
DONE

STEP 7
Count procedures
↓
DONE

STEP 8
Separate procedure blocks
↓
DONE

STEP 9
Extract every procedure
↓
DONE

STEP 10
Create output.json
↓
RUN main.py
↓
Python automatically creates output.json
↓
Open output.json
↓
VERIFY
↓
Only then → STEP 11
```

Direct answer: NO, do not create the JSON file yourself. Put the Step 10 code into main.py and run it. Python creates output.json automatically.

Step 11 — Verify Generated output.json

Purpose: This step checks whether the JSON file created in Step 10 contains the correct data extracted from the PDF. This is a verification step only. Do not change the parser or Step 10 code unless a verification check actually fails.

1. Before Starting

Step 10 should already be completed successfully. You should have an **output.json** file inside the project folder.

```
samarth/  
■■■ main.py  
■■■ pdf_parser.py  
■■■ input/  
■   ■■■ input.pdf  
■■■ output.json  
■■■ PyMuPDF-1.18.18
```

2. Do NOT Change the Code

For Step 11, do not change **pdf_parser.py** and do not change the Step 10 version of **main.py**. The purpose here is simply to inspect and verify the generated JSON.

3. Open output.json

In PyCharm, open **output.json**. The file should contain a JSON list containing one dictionary/object for each procedure.

The first procedure should look similar to:

```
{  
  "Procedure": "abbnm",  
  "Time": "31-dec-2025 11:44:13.103",  
  "Dur": "203.96(s)",  
  "status": {  
    "success": ["01.0001"],  
    "failure": ["01.0002"]  
  }  
}
```

The exact values should match the corresponding information in your actual PDF. The example above is based on the sample procedure already tested.

4. Check the Procedure

Find the first procedure in the original PDF and compare its **Procedure** value with output.json.

Example:

PDF:
Procedure : abbnm

JSON:
"Procedure": "abbnm"

If the values match, the Procedure extraction is correct.

5. Check the Time

Compare the metadata **Time** in the PDF with the **Time** field in output.json.

PDF:

Time:31-dec-2025 11:44:13.103

JSON:

"Time": "31-dec-2025 11:44:13.103"

Make sure the complete timestamp is present, including the milliseconds.

6. Check the Duration

Compare the **Dur** value from the PDF with the **Dur** value in output.json.

PDF:

Dur:203.96(s)

JSON:

"Dur": "203.96(s)"

The value should remain as extracted from the PDF.

7. Check Success and Failure STEP IDs

Inside **status**, check that the STEP IDs are placed under the correct status.

```
"status": {
  "success": [
    "01.0001"
  ],
  "failure": [
    "01.0002"
  ]
}
```

The important point is that the STEP ID must remain exactly as it appears in the PDF. For example, **01.0001** must not become **1.0001** or an integer.

8. Check That Unwanted Data Is NOT Present

The output should contain only the fields required for the project.

Required:

Procedure

Time

Dur

status

 success

 failure

The following unwanted information should not appear in the JSON:

Mode

Source

Priority

STEP description

table TIME

9. Check Several Procedures

Do not check only the first procedure. Scroll through output.json and inspect several procedures.

For each checked procedure, make sure the Procedure, Time, Dur, and status values belong to the same procedure and have not been mixed with another procedure.

10. Check the Number of Procedures

Compare the number of procedure objects in output.json with the procedure count obtained in Step 7 and printed in Step 10.

Example:

Step 7:

Total procedures found: 20

Step 10:

Total procedures: 20

output.json:

20 procedure objects

The numbers should match. The number 20 is only an example; use the actual count from your PDF.

11. Step 11 Success Criteria

Step 11 is successful if all of the following are true:

1. output.json exists.
2. output.json opens normally.
3. The procedure count matches Step 7 and Step 10.
4. Procedure values are correct.
5. Time values are correct.
6. Dur values are correct.
7. STEP IDs are correct.
8. Success STEP IDs are under "success".
9. Failure STEP IDs are under "failure".
10. STEP IDs remain strings such as "01.0001".
11. Unwanted fields are absent.
12. Data from different procedures is not mixed.

12. Important: Do Not Manually Fix the JSON

If a value is wrong, do not manually edit output.json just to make the result look correct. The JSON is generated by the Python program. If the extracted value is wrong, the correct approach is to identify and fix the extraction logic and then regenerate output.json.

13. If Everything Is Correct

If output.json passes all the checks above, Step 11 is complete. Keep the verified output.json as the generated project output.

14. If Something Is Wrong

Stop before moving to the next step. Note exactly what is wrong—for example: a missing Procedure, wrong Time, missing STEP ID, incorrect status grouping, or an incorrect procedure count. Then the parser can be checked based on the actual output.

15. What Comes Next

After Step 11 is successfully verified, the next step is **Step 12 — Edge-Case Testing**. That step checks situations such as multiple Success/Failure STEP IDs, only Success, only Failure, spacing variations, procedures crossing pages, and empty or unusual sections.

Final reminder: Step 11 is only verification. If the generated JSON is correct, do not modify working code.

Step 12 — Edge-Case Testing

Purpose: Step 12 checks whether the PDF parser works reliably in situations other than the exact sample already tested. The goal is to find cases where the parser might miss STEP IDs, assign the wrong status, or incorrectly separate procedures.

1. What You Are Testing

Up to Step 11, the parser has successfully handled the normal sample structure. Now we test different possible data patterns while keeping the expected project output the same.

```
PDF
↓
PyMuPDF
↓
Python parser
↓
Procedure + Time + Dur + status
↓
output.json
```

Do not change the working project just because a test case is different. First observe the result. Only fix the parser if a test reveals an actual problem.

2. IMPORTANT — Do Not Modify Your Real input.pdf

For the tests below, do not overwrite your actual project PDF unless you have a backup. Your current input.pdf is your real dataset.

The safest approach is to create small temporary test PDFs containing the required text patterns. These tests are only for checking the parser logic.

3. Test Case 1 — Multiple Success STEP IDs

The first test checks whether several successful STEP IDs are all placed inside the **success** list.

```
Procedure : test_process
Time:01-jan-2026 10:00:00.000
Dur:100.00(s)
STEP ID          STEP          STATUS
01.0001          test          Success
01.0002          test          Success
01.0003          test          Success
```

Expected status structure:

```
"status": {
  "success": [
    "01.0001",
    "01.0002",
    "01.0003"
  ]
}
```

All three STEP IDs should appear under **success**.

4. Test Case 2 — Multiple Failure STEP IDs

```
Procedure : test_process
```

Time:01-jan-2026 10:00:00.000

Dur:100.00(s)

STEP ID	STEP	STATUS
01.0001	test	Failure
01.0002	test	Failure
01.0003	test	Failure

Expected:

```
"status": {
  "failure": [
    "01.0001",
    "01.0002",
    "01.0003"
  ]
}
```

All three STEP IDs should appear under **failure**.

5. Test Case 3 — Mixed Success and Failure

Procedure : test_process

Time:01-jan-2026 10:00:00.000

Dur:100.00(s)

STEP ID	STEP	STATUS
01.0001	test	Success
01.0002	test	Failure
01.0003	test	Success
01.0004	test	Failure

Expected:

```
"status": {
  "success": [
    "01.0001",
    "01.0003"
  ],
  "failure": [
    "01.0002",
    "01.0004"
  ]
}
```

6. Test Case 4 — Only Success

Procedure : test_process

Time:01-jan-2026 10:00:00.000

Dur:100.00(s)

STEP ID	STEP	STATUS
01.0001	test	Success
01.0002	test	Success

Check whether the parser correctly creates the success list and does not incorrectly assign the STEP IDs to failure.

7. Test Case 5 — Only Failure

Procedure : test_process

Time:01-jan-2026 10:00:00.000

Dur:100.00(s)

STEP ID	STEP	STATUS
01.0001	test	Failure

01.0002

test

Failure

Check whether the parser correctly creates the failure list and does not incorrectly assign the STEP IDs to success.

8. Test Case 6 — Different Spacing

PDF extraction can sometimes contain different amounts of whitespace. Test whether the regular expression still handles normal spacing variations.

```
Procedure      :    test_process
Time : 01-jan-2026 10:00:00.000
Dur : 100.00(s)
```

01.0001	test	Success
01.0002	test	Failure

The important result is that the required values should still be detected when ordinary spaces are present.

9. Test Case 7 — STEP IDs Must Remain Strings

Check the JSON carefully. STEP IDs such as **01.0001** must remain strings.

Correct:
"01.0001"

Incorrect:
1.0001

Do not convert STEP IDs to numbers. The leading zero is part of the STEP ID.

10. Test Case 8 — More Than One Procedure

Test two procedure blocks together. This checks whether the parser keeps the information belonging to each procedure separate.

```
Procedure : process_one
Time:01-jan-2026 10:00:00.000
Dur:100.00(s)
01.0001 test Success
01.0002 test Failure
```

```
Procedure : process_two
Time:01-jan-2026 11:00:00.000
Dur:200.00(s)
02.0001 test Success
02.0002 test Failure
```

Expected: one JSON object for process_one and a separate JSON object for process_two. STEP IDs from process_one must not appear inside process_two.

11. Test Case 9 — Procedure Information Across Pages

Your real PDF may contain a procedure whose information continues onto another page. The project reads the complete PDF into **all_text** before splitting procedures.

Therefore, this case is important to observe. If the real PDF has procedures crossing page boundaries, check whether the generated JSON still contains the complete procedure.

Do not manually change the parser just because a procedure crosses a page. Test the actual PDF behavior first.

12. Test Case 10 — Empty or Unusual Section

Check what happens when a procedure has missing or unusual information. For example, a procedure may have no matching STEP rows.

```
Procedure : test_process
Time:01-jan-2026 10:00:00.000
Dur:100.00(s)
```

The important point is to observe the result. Do not manually invent a status value that was not present in the PDF.

13. What Counts as a Successful Test?

For each test, compare the parser output with what the test input actually contains.

```
Check:
1. Correct Procedure
2. Correct Time
3. Correct Dur
4. Correct STEP IDs
5. Correct success grouping
6. Correct failure grouping
7. No unwanted fields
8. Different procedures remain separate
```

14. If a Test Fails

Do not manually edit output.json to hide the problem. Record exactly what happened.

Example:

```
Expected:
success = ["01.0001", "01.0003"]
```

```
Actual:
success = ["01.0001"]
```

That means the parser did not extract one of the expected STEP IDs. Stop there and investigate the actual extracted PDF text and the regular expression before making a change.

15. Do Not Add Random Features During Testing

Step 12 is only for testing the extraction logic. Do not add AI, OCR, machine learning, databases, web APIs, dashboards, or other features at this stage.

The current project requirement is still the straightforward PDF-to-structured-JSON pipeline.

16. Final Step 12 Checklist

- Multiple Success STEP IDs tested
- Multiple Failure STEP IDs tested
- Mixed Success/Failure tested
- Only Success tested
- Only Failure tested
- Spacing variation tested
- STEP IDs checked as strings

- Multiple procedures tested
- Page-boundary behavior checked
- Empty/unusual section observed
- No unwanted fields introduced
- Any failure recorded instead of manually hiding it

17. When Step 12 Is Complete

If the parser behaves correctly for the relevant test cases, Step 12 is complete.

Then move to **Step 13 — Final Clean Project**. In that step, we will separate temporary testing from the final project and make sure the folder contains only what you actually need.

18. Important — When Will You Learn How to Use It Every Time?

The complete everyday usage procedure will be explained **after Step 12**, during the final-project steps. You will learn exactly how to take another PDF, place it in the input folder, run the program, and obtain the new output.json.

That way, we first prove that the parser works, then clean the project, and finally learn the normal repeatable workflow.

Important: Do not replace your real input.pdf with test data unless you have a backup. The test examples in this guide are for controlled testing only.

Step 13 — Final Clean Project

Purpose: Step 13 is the final cleanup stage. We are not adding new functionality yet. We are taking everything that worked in Steps 1–12 and making the project clean and ready to use.

13.1 — Do NOT delete anything yet

Before cleaning anything, make sure:

- Step 10 generated output.json
- Step 11 verification was successful
- Step 12 testing is completed

If all three are okay, continue.

Do not delete input.pdf.

13.2 — Keep pdf_parser.py

Your pdf_parser.py should contain the extraction functions.

The current working version is:

```
import re

def extract_process_info(text):
    Procedure = ""
    Time = ""
    Dur = ""

    match = re.search(r"Procedure\s*:\s*(.*?)\s*(?=Time\s*:$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Procedure = match.group(1).strip()

    match = re.search(r"Time\s*:\s*(.*?)\s*(?=Mode\s*:$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Time = match.group(1).strip()

    match = re.search(r"Dur\s*:\s*(.*?)\s*(?=Priority\s*:$)", text, re.IGNORECASE | re.MULTILINE)
    if match:
        Dur = match.group(1).strip()

    return Procedure, Time, Dur

def extract_status(text):
    status_data = {}
    lines = text.splitlines()

    for line in lines:
        match = re.match(r"^s*(\d+\.\d+)\s+.*?\b(Success|Failure)\s*$", line, re.IGNORECASE)

        if match:
            STEP_ID = match.group(1)
            STATUS = match.group(2).lower()

            if STATUS not in status_data:
                status_data[STATUS] = []

            status_data[STATUS].append(STEP_ID)

    return status_data
```

Do not change this just for the sake of cleaning.

13.3 — Final main.py

Now your final main.py should be the version that reads the complete PDF and generates output.json.

```
import fitz
import re
import json

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""

for page in pdf:
    all_text = all_text + page.get_text() + "\n"

procedure_blocks = re.split(r"(?=Procedure\s*:)", all_text, flags=re.IGNORECASE)
procedure_blocks = [block.strip() for block in procedure_blocks if block.strip()]

all_processes = []

for block in procedure_blocks:
    Procedure, Time, Dur = extract_process_info(block)
    status_data = extract_status(block)

    process_data = {
        "Procedure": Procedure,
        "Time": Time,
        "Dur": Dur,
        "status": status_data
    }

    all_processes.append(process_data)

with open("output.json", "w") as file:
    json.dump(all_processes, file, indent=4)

print("Total procedures:", len(all_processes))
print("JSON file created successfully.")

pdf.close()
```

This is now the **normal final program**.

13.4 — Remove temporary testing code

During Steps 6–8, you used temporary versions of main.py, such as:

```
print("Total pages:", len(pdf))

print("Total procedures found:", len(procedures))

for i, block in enumerate(procedure_blocks, start=1):
```

Those were only for testing.

You do not need separate test programs for the final project.

Your final main.py is the Step 10 version above.

13.5 — What about output.json?

Keep it.

You don't need to manually create it.

Every time this line runs:

```
with open("output.json", "w") as file:
```

Python creates or overwrites the file.

First run

↓

output.json created

Second run

↓

output.json updated/replaced with new result

You don't need to manually delete it every time.

13.6 — Check your folder

Your project should now look like:

```
samarth/
■
■■■ main.py
■
■■■ pdf_parser.py
■
■■■ input/
■   ■■■ input.pdf
■
■■■ output.json
■
■■■ PyMuPDF-1.18.18
```

Keep:

- ✓ main.py
- ✓ pdf_parser.py
- ✓ input/
- ✓ input.pdf
- ✓ output.json
- ✓ PyMuPDF folder/environment files required by your setup

Don't keep:

- ✗ Temporary test PDFs
- ✗ Random test .py files
- ✗ Old experimental parser files
- ✗ Unnecessary temporary output files

13.7 — Run the final project once

Now run:

```
main.py
```

You should get something like:

```
Total procedures: 20
```

JSON file created successfully.

The number 20 is only an example. Your actual number should be the number of procedures in your PDF.

13.8 — Open output.json

After running main.py, open output.json.

Check that it still contains the expected structure:

```
[
  {
    "Procedure": "abbnm",
    "Time": "31-dec-2025 11:44:13.103",
    "Dur": "203.96(s)",
    "status": {
      "success": [
        "01.0001"
      ],
      "failure": [
        "01.0002"
      ]
    }
  }
]
```

Your actual file will contain all the procedures.

13.9 — Very important: Don't test with a random PDF yet

At this stage, do not replace input/input.pdf with another random PDF.

Your current PDF is the dataset we have already been testing.

The next step will explain exactly how to use the program with another PDF.

13.10 — What Step 13 accomplishes

Before Step 13:

```
Testing
↓
temporary code
↓
checking
↓
experiments
```

After Step 13:

```
Clean final project
↓
main.py
↓
pdf_parser.py
↓
input PDF
↓
output.json
```

So from this point onward, we're treating this as the **actual project**, not a collection of experiments.

Step 13 Checklist

Before saying Step 13 is complete, check:

- `main.py` contains the final full-PDF code
- `pdf_parser.py` contains the working extraction functions
- `input/input.pdf` exists
- `output.json` exists
- Temporary test files are removed
- `main.py` runs without an error
- "JSON file created successfully." appears
- Procedure count is correct
- `output.json` contains the expected structure
- No unwanted fields are present

After Step 13

The next step is the important one you asked about earlier:

Step 14 — How to Use This Every Time With a New PDF

There we'll make it very practical:

"I received a new PDF. What exactly do I do from start to finish?"

We'll cover where to put it, what filename to use, what to run, what gets overwritten, where the result appears, and how to use the generated JSON.

Important: Step 13 is cleanup and finalization only. Do not add new project features here.

Step 14 — How to Use the Project Every Time

Purpose: This step explains the normal usage of your finished program. You do not repeat Steps 6–12 every time. Those were development and testing steps.

14.1 — Your Current Folder

```
samarth/  
■■■ main.py  
■■■ pdf_parser.py  
■■■ input/  
■   ■■■ input.pdf  
■■■ output.json  
■■■ PyMuPDF-1.18.18
```

For now, don't create an archive folder and don't change the structure.

14.2 — When You Get a New PDF

Suppose you receive a PDF named **report.pdf**.

Step 1: Put the PDF inside the **samarth/input/** folder.

Step 2: Rename it to **input.pdf**.

```
samarth/  
■■■ input/  
    ■■■ input.pdf
```

This is necessary because **main.py** currently contains:

```
pdf = fitz.open("input/input.pdf")
```

14.3 — Run the Program

Open **main.py** and run it.

You should see something like:

```
Total procedures: 20  
JSON file created successfully.
```

The number will depend on your PDF.

14.4 — Check the Result

After the program finishes, open **output.json**.

It will contain the information extracted from your PDF, for example:

```
[  
  {  
    "Procedure": "abbnm",  
    "Time": "31-dec-2025 11:44:13.103",  
    "Dur": "203.96(s)",  
    "status": {  
      "success": [  
        "01.0001"  
      ],  
      "failure": [  
        "01.0002"  
      ]  
    }  
  }  
]
```

```

    ]
  }
}
]

```

Your actual output will contain the data from your actual PDF.

14.5 — What Happens to the Previous output.json?

You do not need to delete it manually.

When main.py runs the following code:

```
with open("output.json", "w") as file:
```

Python replaces the previous output.json with the new result.

```

Old PDF
↓
Old output.json

Replace input.pdf with new PDF
↓
Run main.py
↓
New output.json

```

14.6 — What Happens to the Previous PDF?

For the simple setup we are using now, there is only one PDF inside the project: **input/input.pdf**.

When you want to process a new PDF, replace the old input.pdf with the new PDF and rename the new one to input.pdf.

If you may need the old PDF or old JSON later, copy it somewhere else on your computer before replacing it. For now, we are not creating an archive system.

14.7 — What If There Is an Error?

If PyCharm shows an error, do not randomly change the code. Look at the error message.

For example, **FileNotFoundError** may mean that the PDF is not correctly located at **input/input.pdf**.

If you get a parser or extraction problem, keep the PDF and generated output and check the problem before changing the code.

14.8 — What If There Is No Error but the JSON Is Wrong?

Check output.json. For example, if the PDF contains **Procedure : abbnm** but the JSON contains an empty Procedure, the program ran but extraction was wrong.

Do not manually edit the JSON. Show the result and investigate the actual extraction problem.

14.9 — The Only Workflow You Need to Remember

```

1. Get PDF
↓
2. Put it in input/
↓
3. Rename it to input.pdf
↓

```

4. Run `main.py`
↓
5. Open `output.json`
↓
6. Check the extracted data

You do not need to run Steps 7–12 separately, manually create JSON, manually enter extracted information, or repeat edge-case testing every time. Those were only for building and verifying the project.

Step 14 Checklist

- Put the PDF in `input/`
- Rename it to `input.pdf`
- Run `main.py`
- Check "JSON file created successfully."
- Open `output.json`
- Check the extracted information

One-line explanation: Put the PDF in `input/input.pdf`, run `main.py`, and the extracted result will be generated in `output.json`.

Step 14 is now complete.

Next: Step 15 — Final Project Workflow + how to explain the project to your instructor.